Edge Caching Optimization With PPO and Transfer Learning for Dynamic Environments

Farnaz Niknia and Ping Wang, *Fellow, IEEE*

***Abstract*—This article addresses the challenge of edge caching in dynamic environments, where rising traffic loads strain backhaul links and core networks. We propose a proximal policy optimization (PPO)-based caching strategy that fully incorporates key file attributes, such as size, lifetime, importance, and popularity, while also considering random file request arrivals, reflecting more realistic edge caching scenarios. In dynamic environments, changes, such as shifts in content popularity and variations in request rates frequently occur, making previously learned policies less effective as they were optimized for earlier conditions. Without adaptation, caching efficiency and response times can degrade. While learning a new policy from scratch in a new environment is an option, it is highly inefficient and computationally expensive. Thus, adapting an existing policy to these changes is critical. To address this, we develop a mechanism that detects changes in content popularity and request rates, ensuring timely adjustments to the caching strategy. We also propose a transfer learning-based PPO algorithm that accelerates convergence in new environments by leveraging prior knowledge. Simulation results demonstrate the significant effectiveness of our approach, outperforming a recent deep reinforcement learning (DRL)-based method.**

***Index Terms*—Change detection, edge caching, proximal policy optimization (PPO), semi-Markov decision process (MDP), transfer learning.**

I. Introduction

With the rapid growth of mobile applications and services, the demand for data has surged, placing significant pressure on backhaul links and core networks. The exponential rise in video streaming, cloud services, and Internet of Things (IoT) devices has led to an overwhelming number of content requests, often from centralized data centers. To address this, edge caching frequently requested data closer to users has become a key strategy for reducing network congestion and minimizing transmission delays. By caching popular content locally, edge caching not only alleviates the load on core networks but also improves the overall user experience by reducing latency.

Edge caching faces significant challenges in dynamic and unpredictable environments, where content popularity is not static and fluctuates based on factors like time, location, and social trends [1], [2], [3], [4]. A viral video, for instance, may experience a sudden spike in requests, followed by a rapid decline in demand. Similarly, request rates can change unpredictably, influenced by factors, such as user mobility, device connectivity, and network congestion. For instance, during peak hours in a densely populated area, a surge in user activity may dramatically increase the request rate for certain content. On the other hand, during off-peak times, the request rate may drop significantly. Events like sports games or breaking news can also cause rapid shifts in content requests. When it comes to dynamic environments, most existing research primarily focuses on changes in content popularity trends while overlooking the equally important fluctuations in request rates. In real-world scenarios, users' behavior and usage patterns can shift over time, which directly influences the rate at which files are requested. For instance, during peak hours, when many users are accessing the network at once, demand for content rises sharply. This increased activity results in shorter intervals between file requests, as users require quick access to different resources. In contrast, during off-peak periods, when fewer users are online, the intervals between file requests tend to lengthen. With reduced demand for immediate access, users are less likely to request content frequently, leading to extended gaps between consecutive requests.

Existing caching strategies, particularly those relying on fixed policies, often fail to account for these dynamic shifts, leading to inefficient use of cache resources. Therefore, the ability to learn new policies quickly and efficiently becomes crucial for maintaining high cache performance in such environments.

Most of the existing work on reinforcement learning (RL) in dynamic environments assumes that changes occur gradually, giving the agent sufficient time to adapt its policy incrementally. This assumption, however, is not always valid, especially in real-world scenarios where sudden shifts in content popularity or request rates are common. In such cases, relying on gradual adaptation may lead to poor performance. Therefore, detecting when these abrupt changes happen becomes critical to maintaining high system efficiency. By quickly identifying the moment of change, essential steps can be taken to adjust the RL policy and prevent the agent from making decisions based on outdated information. Detecting these shifts promptly allows the agent to recalibrate and ensure that caching decisions remain relevant.

While training caching policies from scratch after every environmental change is one option, it is not practical. Learning a new policy from the ground up each time a

Received 14 November 2024; revised 24 May 2025; accepted 28 May 2025. Date of publication 9 June 2025; date of current version 8 August 2025. This work was supported by a Canada NSERC Discovery Grant. *(Corresponding author: Ping Wang.)*

The authors are with the Lassonde School of Engineering, York University, Toronto, ON M3J 1P3, Canada (e-mail: fniknia@yorku.ca; ping.wang@lassonde.yorku.ca).

Digital Object Identifier 10.1109/JIOT.2025.3577904

2327-4662 © 2025 Crown Copyright

Authorized licensed use limited to: York University. Downloaded on November 03,2026 at 23:52:02 UTC from IEEE Xplore. Restrictions apply.

shift occurs would result in significant delays, during which the system operates suboptimally. This can lead to increased latency and reduced cache efficiency. For this reason, it is important to converge to the optimal policy faster in the new environment, as this ensures the system can restore its high level of performance without unnecessary delays or inefficiencies.

To address these limitations, we introduce a caching strategy based on proximal policy optimization (PPO) [5], an RL technique known for its stability and efficiency. In our approach, we incorporate not only content popularity but also key file attributes, such as size, lifetime, and importance to ensure that the cache is used optimally.

In this work, we propose a mechanism for detecting changes in content popularity and request rates, designed to identify shifts in the environment quickly. This mechanism has two functions. The first function focuses on popularity change detection by leveraging cosine similarity between recent and past request patterns. This method effectively captures shifts in content popularity by comparing how closely current requests align with historical trends. The second function addresses request rate changes using a simple moving average over a defined window of time. By continuously monitoring and averaging request rates, it quickly detects deviations from the norm, signaling a change in the environment. Both functions are designed to detect changes rapidly, allowing the system to adapt swiftly and maintain optimal performance in dynamic scenarios.

To quickly adapt to changing environments, we integrate transfer learning into the PPO algorithm by transferring the learned knowledge in the form of transitions to the new environment. In the new environment, we assign priorities to both the transferred state transitions and the new transitions collected from the current environment. These priorities are determined using a combination of the temporal difference (TD) error associated with each transition and the difference between the average reward and the instant reward tied to that transition. By assigning higher priority to more relevant transitions, the caching agent is able to focus on learning from the most significant experiences, thus accelerating convergence. This approach is particularly advantageous in dynamic edge caching environments, where frequent changes demand swift adaptation, and leveraging prior knowledge minimizes the time and computational resources required for the agent to adjust to new conditions.

In summary, our approach tackles the limitations of traditional caching strategies by dynamic change detection and transfer learning. This results in a caching system that is flexible, efficient, and capable of adapting quickly to changing environments, ensuring optimal performance even in highly dynamic scenarios. The contributions of this article are summarized as follows.

1) We introduce a mechanism with two functions specifically designed to rapidly detect changes in the environment. These functions are engineered to enhance the responsiveness of the system to dynamic shifts in content popularity and request rate, ensuring timely and accurate identification of environmental changes.
2) We propose a novel algorithm that combines transfer learning with PPO. This innovative approach facilitates accelerated adaptation to new environments by effectively leveraging previously acquired knowledge, thereby enhancing the algorithm's ability to quickly adjust and optimize caching strategies in dynamic settings.
3) We conduct simulations where the environment undergoes sudden changes, and our change detection approaches demonstrated remarkable speed in identifying these change points. Furthermore, our proposed transfer learning algorithm outperforms several benchmark methods, including learning from scratch (LFS), learning from demonstrations [6], and another recently published work [7]. This superiority highlights the effectiveness of our approach in rapidly adapting to new conditions and achieving better performance in dynamic environments.

The remainder of this article is structured as follows. Section II provides an overview of related work. Section III introduces our system model, while Section IV outlines our proposed caching algorithm. In Section V, we discuss our experimental setup and results, and Section VI concludes this article. Table I summarizes the most important notations used in this article.

II. Related Work

The literature on caching strategies can broadly be categorized into two main approaches based on how they handle file popularity: 1) those assuming static popularity and 2) those addressing dynamic popularity. In the former, file request patterns are considered fixed over time, with algorithms optimized to perform under the assumption that popular content remains consistent. On the other hand, the latter body of work focuses on environments where file popularity changes over time, necessitating adaptive caching strategies that can respond to shifting user demand. The following sections will review research under these two categories.

A. Caching With Static File Popularity

Article [8] tackles the challenge of efficient edge caching in multitier computing networks, where computing, caching, and networking resources are distributed from the cloud to users through edge servers. The study addresses the high-cost nature of direct cache hits caused by user content requests. To overcome this, the authors propose integrating recommender systems with edge caching in mobile two-tier (edge-cloud) networks to support both direct and soft hits, improving the resource utilization of edge servers. They model the problem as a Markov decision process (MDP) and introduce a decentralized caching framework using a multiagent actor-critic algorithm and federated learning. This enables edge servers to independently learn optimal caching strategies while minimizing long-term system costs.

Study [9] addresses the challenges in IoT networks related to transient data generation, limited energy resources, and Quality of Service (QoS) requirements by proposing a deep

Authorized licensed use limited to: York University. Downloaded on November 03,2026 at 23:52:02 UTC from IEEE Xplore. Restrictions apply.

TABLE I
TABLE OF NOTATIONS

Symbol	Description
f_r	the requested file
F	The total number of file types
f_f	file type f
λ	The request rate
η	Skewness of popularity distribution
N	length of the sliding window to store the history of the file requests
γ	The discount factor
$\|\Lambda\|$	The size of the replay buffer before the change in the environment
$\|\Lambda'\|$	The size of the replay buffer after the new environment
β	Controls the importance sampling
d_f	The number of requests for file type f over the last N requests
l_f	Lifetime of file type f
i_f	Importance of file type f
z_f	Size of file type f
$h^f(t)$	Freshness of file type f
$y_f(t)$	Utility of file type f
b	The files cached in the edge router's cache
π	Policy
θ	Weights of the actor network
θ'	Weights of the critic network
s	Current state
s'	Next state
$r(t)$	The instant reward at time t
τ	State transtition time
r_i	The instant reward of transition i
$r_{(n)}$	The immediate reward at the nth step
p_i	The priority of sampling transition i
$P(i)$	The probability of sampling transition i
L_P	The size of the moving average window for popularity change detection
L_R	The size of the moving average window for request rate change detection
th_R	Threshold for the acceptable variance of the request rate
th_c	Threshold for the acceptable variance of the cosine similarity

RL (DRL)-based caching strategy. The authors design a hierarchical architecture caching scheme that optimizes both cache hit rate and energy consumption while considering data freshness and the limited lifetime of IoT data.

Zhou et al. [10] explore the integration of edge computation offloading and content caching in 6G-enabled Internet of Vehicles (IoV) to tackle the computational and communication demands of emerging vehicular applications. The proposed framework leverages the synergy between computation offloading and caching to reduce latency, improve service quality, and optimize resource usage in highly dynamic IoV environments.

To address challenges in caching transient IoT data at the edge, researchers in [11] proposed a DRL-based approach that dynamically manages IoT data caching without prior knowledge of data popularity or user request patterns. The approach considers data freshness as a key metric to balance communication costs and freshness loss, which is essential in IoT applications, such as autonomous vehicles, where data must remain timely and accessible.

The approach proposed in [9] improves cache hit rates and reduces energy consumption without requiring prior knowledge of data patterns, tackling issues like data freshness and limited data lifetimes. The scheme uses a hierarchical architecture to capture regional variations in data popularity by deploying edge caching nodes strategically within IoT networks.

The research work presented in [12] proposes a novel framework to enhance content delivery in edge-cloud networks by integrating content recommendation systems with federated multiagent DRL. The authors model the caching and delivery decision-making process as a partially observable MDP (POMDP) and employ deep deterministic policy gradient (DDPG) to learn optimal policies. To address the challenges of model personalization and local optima in multiagent scenarios, they develop a federated distributed training strategy that incorporates Earth mover's distance (EMD) for personalized model aggregation.

Wang et al. [13] introduce a DRL-based framework for intelligent edge caching, aimed at optimizing cache hit rates, transmission delay, and energy efficiency. This framework leverages a novel content popularity model inspired by Newton's law of cooling to dynamically assess content relevance over time, integrating content size and access patterns into the caching decision process via deep Q-networks (DQN). To enhance scalability and caching efficiency across multiple edge nodes, the authors further propose a distributed cooperative caching scheme that employs consistent hashing and a multitier node architecture.

Both recent works [12] and [13] assume a static popularity distribution for content and a constant request rate, failing to account for the dynamic nature of user behavior in real-world scenarios. In practical environments, content popularity fluctuates over time due to changing user interests, events, and trends, while request rates often vary based on time of day, location, and user mobility. This static assumption limits the applicability and effectiveness of the proposed methods when deployed in real systems, where adaptability to changing conditions is crucial for maintaining optimal performance.

B. Caching With Dynamic File Popularity

Zhong et al. [14] propose a DRL approach to dynamically adapt caching policies to time-varying content popularity. Using the Wolpertinger architecture, they design actor–critic DRL frameworks for both centralized and decentralized caching scenarios. The centralized policy focuses on maximizing the cache hit rate, while the decentralized approach optimizes both the cache hit rate and transmission delay.

Study in [15] addresses the issue of managing heavy backhaul traffic generated by user-generated videos, which are characterized by a surge in demand within a short time frame after release. The authors propose a solution that combines content popularity prediction and reactive caching to optimize resource utilization in a three-tier wireless network. To predict popularity in various locations, they employ a recommendation system that translates request probabilities into rating scores. Using these predictions, a DRL-based caching strategy is introduced to make timely caching decisions for newly requested content, aiming to maximize cache efficiency while minimizing storage costs.

Authorized licensed use limited to: York University. Downloaded on November 03,2026 at 23:52:02 UTC from IEEE Xplore. Restrictions apply.

Study [16] focuses on optimizing the content placement strategy, aiming to minimize data transmission delay while considering cache storage limitations and data freshness constraints. This problem is formulated as an integer linear programming (ILP) model and later as an MDP, and the authors apply a DRL algorithm to address the dynamic nature of the caching problem.

Study [17] addresses the problem of caching in ultradense networks with wireless backhaul, focusing on optimizing caching strategies at small base stations (SBSs) with limited storage. The challenge arises from the dynamic and unknown variations in content popularity over time. The authors propose a cooperative caching strategy that leverages RL combined with maximum-distance separable coding to maximize the traffic load SBSs serve, avoiding reliance on the macro base station. The problem is formulated as an MDP, and Q-learning is applied to small-scale systems. They introduce a heuristic approximation of the state-action value function for large-scale systems, significantly reducing the complexity of action selection.

Fan et al. [18] propose an evolving learning-based content caching policy that adaptively learns the changing content popularity and decides which content to replace when the cache is full. This approach uses a multilayer recurrent neural network (RNN) that evolves its training process from shallow to deeper layers as more requests arrive, offering a dynamic and scalable solution.

Study [4] addresses the problem of designing an optimal coded caching policy in wireless networks to minimize the total network cost. To tackle the unknown and time-varying content popularity and the continuous, high-dimensional action space, the authors propose a clustering-based long short-term memory (C-LSTM) approach to predict the number of content requests using historical data, leveraging correlations between different files. Based on these predictions, they develop a supervised DDPG method to learn the caching policy in a continuous action space using an actor–critic architecture.

Zong et al. [2] address the issues of dynamic content popularity and heterogeneous caching configurations. The authors propose an ensemble learning approach that adapts to varying caching scenarios by selecting from multiple caching policies, rather than relying on a single dominant policy. They analyze the complementary strengths of least frequently used (LFU) and least recently used (LRU) policies in different situations and develop a caching algorithm that enhances these policies using a deep RNN (LSTM) for time-series analysis. Additionally, they introduce a DRL agent that dynamically combines base caching policies based on their virtual hit ratios.

Gao et al. [1] focus on designing a dynamic probabilistic caching strategy for scenarios where content popularity fluctuates over time, but the average content popularity over a time window can be predicted. The goal is twofold: 1) to converge to optimal caching probabilities when content popularity is stable and 2) to adapt to time-varying popularity when it changes. The authors propose a dynamic content replacement method that addresses the limitations of static caching, which cannot adapt, and traditional dynamic policies like LRU, which fail to converge to optimal caching probabilities. The problem is modeled to find the state transition probability matrix of a Markov chain, and a method is developed to generate and refine this matrix.

Study in [19] explores content caching in wireless small-cell networks, where popular files are stored at SBSs to reduce load on the macro cellular network. The authors consider a scenario where popularity varies based on factors, such as changing user types. To solve this, they propose a caching algorithm based on contextual multiarmed bandits, allowing the SBS to update its cache dynamically and learn context-dependent popularity profiles over time.

Zhou et al. [7] propose a Q-value-based algorithm for joint radio and cache resource allocation in 5G networks, addressing the need for more efficient and flexible resource management. This approach incorporates knowledge transfer into the double deep Q-network framework, enabling agents to leverage expert knowledge, which accelerates learning and reduces the training burden.

C. Motivation

While there are research works addressing dynamic content popularity, these works assume a fixed request rate, meaning that the rate at which requests are generated remains constant over time. However, this assumption is not practical for real-world deployments where the request arrival patterns can vary unexpectedly. Ignoring the impact of changing interarrival times can result in caching policies that are optimized for specific request patterns but become ineffective when the temporal dynamics shift. This can lead to increased latency and reduced cache hit rate as the system is unable to adapt to the altered demand patterns.

To the best of our knowledge, there is no research in the literature that explicitly addresses the problem of varying request rates in edge caching systems. This is a critical gap because even a caching strategy that adapts to changes in content popularity will struggle when the rate at which requests arrive deviates from what the policy has been trained on. For example, if a policy is tuned for frequent requests but is suddenly faced with infrequent arrivals, it may result in excessive cache replacements or underutilization of resources. Conversely, if the request rate increases significantly, the policy might fail to keep pace with the higher demand, leading to cache saturation and performance degradation.

Therefore, it is essential to consider both dynamic content popularity and varying request rates when designing edge caching policies. A caching strategy that is sensitive to request rate fluctuations can make more informed decisions about when to replace cached content or prefetch new files, ensuring optimal performance under diverse traffic conditions. By capturing both the content dynamics and temporal variations in request patterns, such an approach would be more robust and better suited for real-world edge caching applications, thereby improving cache hit rate, reducing latency, and minimizing the overall network load. This study aims to address this gap and propose a novel solution that effectively handles varying request rates, ensuring adaptive and efficient caching performance in dynamic environments.

Authorized licensed use limited to: York University. Downloaded on November 03,2026 at 23:52:02 UTC from IEEE Xplore. Restrictions apply.

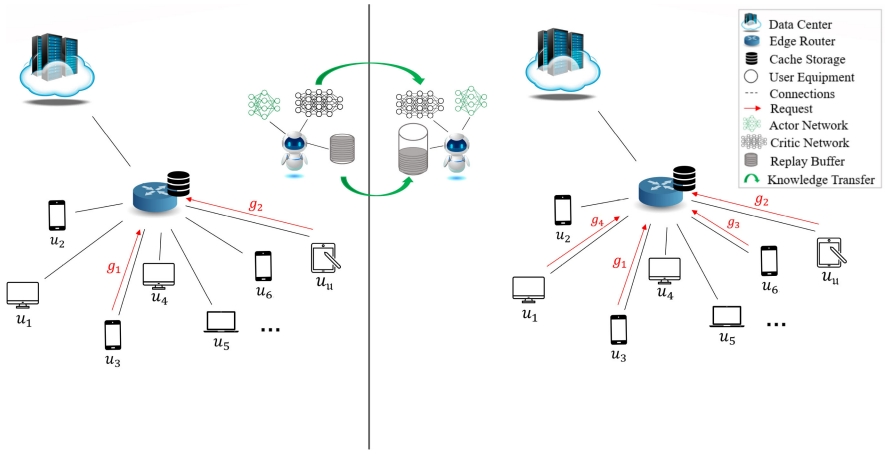


Fig. 1. Caching system topology. In the left figure, users u_1, u_2, u_3, and u_u are connected to the router. Users u_3 and u_u send requests g_1 and g_2 respectively. The right figure shows the same system but after an increase in request rate.

Moreover, most of the existing research works do not involve the detection of changes in either content popularity or request rate. Instead, they assume that the exact time points where these changes occur are known beforehand. This assumption simplifies the design of caching strategies, as the models can directly adjust their policies based on predefined change points. However, in practical scenarios, the timing and nature of these changes are typically unknown and can occur unpredictably. Relying on prior knowledge of such changes is unrealistic and limits the applicability of these strategies in real-world environments, where both content popularity and request rates can fluctuate unpredictably. This motivates us to tackle the issue of environmental change detection in our work.

III. System Model

A. System Architecture

We analyze a network environment where cache storage resides within the edge router, which is directly connected to the end users on one side and the data center on the other side, as shown in Fig. 1. The capacity of the edge router is denoted as M.

It is assumed that the cloud data center possesses sufficient capacity to store all content [20]. Upon receiving a file request from an end user, a copy is generated and transmitted to the user via the edge router over the Internet. If the requested file f_r is already cached in the edge router, the request is fulfilled directly from its cache without retrieving the file from the data center. From this point forward, the term "cache" refers to the edge router's cache. The end users, represented as devices, request files based on their respective needs and preferences [20]. Let the set of users connected to the edge router be denoted by $\mathcal{U} = \{u_1, u_2, \ldots, u_U\}$. Requests are represented by $\mathcal{G} = \{g_1, g_2, \ldots, g_G\}$, where g_g represents the gth request, irrespective of which user initiates it. Requests are processed in a first-come-first-served manner.

Files, generated from sources, such as cameras, sensors, and computers, are stored within the data center. Let $\mathcal{F} = \{f_1, f_2, \ldots, f_F\}$ denote the set of file types, where f_f represents the fth file type. Each file type may exhibit different characteristics, including popularity, lifetime, size, and importance. These features can be further explained using real-life examples.

1) *Popularity:* The number of times a file is requested by users [21]. Popular files, such as trending videos, tend to be requested more frequently than less popular ones.
2) *Lifetime:* The period during which a file remains valid after leaving the data center [22]. For example, location-based services often necessitate timely updates, meaning files are assigned a predetermined lifetime upon generation [23].
3) *Size:* Files vary in size depending on their type and content. For instance, a movie file is generally much larger than a text file.
4) *Importance:* Represents the value a file holds for users based on its relevance and necessity. Files of critical importance, such as real-time financial data or emergency updates, are more valuable than less critical files like free e-books or casual videos.

A comprehensive caching policy must account for all file characteristics simultaneously. Focusing on only one attribute might neglect other crucial aspects. For example, a highly popular file with a short lifetime, large size, and low importance may be less beneficial than a less popular file with a longer lifetime, smaller size, and higher importance.

This work considers the above four key characteristics for each file. Let $\mathbf{d}(t) = (d_1(t), d_2(t), \ldots, d_F(t))$ represent the popularity values for each file type, where d_f corresponds to the popularity of file type f. This indicates the number of times file type f is requested within a predefined time interval N. Additionally, the lifetime, size, and importance of each file type at time t are denoted as $\mathbf{l} = (l_1, l_2, \ldots, l_F)$, $\mathbf{z} = (z_1, z_2, \ldots, z_F)$, and $\mathbf{i} = (i_1, i_2, \ldots, i_F)$, respectively.

The utility of a cached file, defined as a function of its freshness and importance, determines its overall value. Freshness refers to the age of a file relative to its lifetime at time t, expressed as

$$h^f(t) = \left(\frac{t - w_g^f}{w_l^f}\right), \quad 0 \le h^f(t) \le 1$$

Authorized licensed use limited to: York University. Downloaded on November 03,2026 at 23:52:02 UTC from IEEE Xplore. Restrictions apply.

where t denotes the current time, w_l^f is the lifetime, and w_g^f is the file f's generation time.

The utility function for file type f at time t is denoted as $y_f(t) = E(h^f(t), i_f)$, where $y_f(t)$ represents the utility of file type f at time t. The function E depends on two essential factors: 1) the file's freshness $h^f(t)$ and 2) the file's importance i_f. The function E captures the interaction between these elements in determining the overall utility. Typically, utility $y_f(t)$ increases with the file's importance i_f, reflecting its greater value to the system. Moreover, there is a direct relationship between a file's freshness and its utility; as freshness $h^f(t)$ decreases, indicating the file is becoming outdated, the utility $y_f(t)$ decreases accordingly. Although the specific definition of a utility function varies across applications and user perspectives, the method proposed here is adaptable and compatible with any utility function definition.

B. System Uncertainties

Caching involves inherent uncertainties, such as unpredictable request arrivals and the unknown impact of future requests on the cache. These uncertainties influence the edge router's decision-making process. Below, we describe the primary uncertainties and our corresponding assumptions.

1) Change in Popularity: In dynamic environments, such as edge caching systems, the popularity of cached content can fluctuate over time. This change in popularity refers to the variation in how often users request certain files or data. Various factors, such as user preferences, temporal trends, and external events, typically influence the popularity distribution. As the popularity changes, files that were previously requested frequently may become less popular, while others gain prominence. Managing this uncertainty is crucial to maintaining optimal cache performance, as the caching policy must continuously adapt to the shifting popularity of content to reduce latency and minimize cache misses.

2) Change in Request Rate: Another major source of uncertainty in our system is the variability in the request rate, which refers to the number of file requests per unit of time. The request rate can fluctuate based on user behavior, network conditions, or time of day, and these fluctuations can impact the performance of the caching system. A sudden spike in requests, for example, may overload the system, leading to increased latency. On the other hand, a reduction in the request rate may result in underutilized cache resources. We model request arrivals using a Poisson process, a common model for characterizing user request patterns [24]. The parameter λ represents the request rate. We assume that the edge router has no prior knowledge of the Poisson process or its parameters.

IV. Proposed Caching Algorithm

In this section, we first formulate the edge caching problem using the semi-MDP (SMDP) framework, capturing the system's dynamics in continuous time and defining its key components, including states, actions, rewards, and dynamics. Following the SMDP formulation, we provide an overview of the PPO algorithm, which serves as the core RL method for optimizing the caching policy. We then introduce our change detection mechanisms designed to identify shifts in both content popularity and request arrival rate, enabling the system to recognize when the environment has changed. Finally, we describe how transfer learning is employed to accelerate policy adaptation after such changes by leveraging knowledge from the previously learned policy, improving convergence speed.

A. SMDP Formalism

The responsibility of the edge router involves deciding which files should be cached, taking into account factors, such as user request frequencies, file properties, and cache memory constraints. This decision-making procedure is inherently sequential, making MDPs a suitable modeling approach, as outlined in [25].

An MDP is defined by five key elements: 1) state; 2) action; 3) system dynamics; 4) reward; and 5) policy. In this context, a state represents the condition of the system at a given time instance. The agent selects an action following a defined policy, transitions into a new state, and receives a reward based on the action's quality. This process repeats either indefinitely (infinite horizon) or until a termination point is reached (finite horizon) [26]. In the caching problem, the infinite-horizon approach is applicable since no predefined termination state exists. As requests arrive continuously, the problem requires a continuous-time extension of the MDP. Therefore, we employ the SMDP framework, which can handle variable transition times [25]. We follow our previous work for SMDP formulation presented in [27]. Below, we briefly explain the components of our SMDP formulation and its components.

An SMDP is characterized by the tuple $(\mathcal{S}, \mathcal{A}, J, R, \pi)$, where $\mathcal{S}$ denotes the state space, $\mathcal{A}$ represents the action space, J signifies transition durations, R is the reward function, and π is the policy.

1) System States: At time t, the system state is given by $s_t \in \mathcal{S}$ and includes

$$s(t) = \{\text{Mem}(t),\ \mathbf{b}(t),\ \mathbf{y}(t),\ \mathbf{d}(t),\ \mathbf{i}(t),\ \mathbf{l}(t),\ \mathbf{z}(t)\}$$

where $\text{Mem}(t)$ represents the available cache memory in the edge router. Their values are defined as follows:

$$\text{Mem}(t) = \left(\frac{M - \sum_{f=1}^{F} \mathbf{b}(t) \cdot \mathbf{z}(t)}{M} \right)$$

where $\mathbf{b}(t)$ is a binary vector, each element indicating whether a specific file is cached in the edge router. This vector is defined as $\mathbf{b}(t) \in B = \{0, 1\}^F$, with 1 representing a cached file and 0 representing an uncached file. The vector $\mathbf{y}(t)$ tracks the utility of cached files in the edge router. IF $y_f(t) = 0$, the file f is not cached. Additionally, $\mathbf{i}(t)$, $\mathbf{l}(t)$, and $\mathbf{z}(t)$ represent the importance, lifetime, and size of each file, respectively.

2) Actions: At any time t, the agent selects one of the two possible actions: $a(t) = 1$ (cache the file in the cache), or $a(t) = 0$ (do not cache the file). If the cache is full, the file with the lowest utility is evicted to create space for new files [28], [29]. Multiple files can be removed if further space is required.

Authorized licensed use limited to: York University. Downloaded on November 03,2026 at 23:52:02 UTC from IEEE Xplore. Restrictions apply.

3) System Dynamics: In this SMDP model, with stochastic task arrivals, the system's evolution depends on both state transition probabilities ($P_{ss'}$) and the associated transition times (τ). The inclusion of time alongside state transitions increases the complexity of decision-making, compared to traditional MDPs. If the system dynamics and the reward function were fully known, Bellman equations could be applied to determine the optimal policy. However, as this information is typically unknown in practical applications, RL is employed to iteratively learn the system dynamics and optimize the policy through experience. The specifics of the RL algorithm used will be described in subsequent sections.

4) Reward: The immediate reward is expressed as

$$r(t) = w_1\left[(\mathbf{b}(t)\cdot\mathbf{d}(t))(\mathbf{b}(t)\cdot\mathbf{y}(t))^T\right] - w_2[Mem(t)]. \tag{1}$$

The first term captures the reward for serving useful content from the cache: it combines the popularity and utility of the cached files. The second term penalizes inefficient caching by subtracting a value proportional to the unused cache space. This ensures that the caching policy favors keeping high-utility files in the cache while minimizing wasted memory. To illustrate the behavior of the reward function in our system, consider two scenarios in which all system components remain constant except for a single varying parameter. In the first scenario, the cache is fully occupied with files that have high utility, meaning they are fresh and important. Since all requested files are present in the cache and their utility is high, the reward function yields a relatively high value. Additionally, because the cache is fully utilized, there is no penalty for unused memory. In the second scenario, assume that some time has passed, causing the utility of the cached files to decline due to reduced freshness. Although the files are still cached and requested, their contribution to the reward decreases, as the utility values are lower. This reflects the system's preference for keeping fresh and relevant content in the cache. The reduction in utility results in a lower reward, even though cache occupancy and request patterns remain the same. In another variation, consider a case where some cache space remains unoccupied. Even if the utility of the cached files is high, the reward will be penalized due to the presence of unused cache memory. This highlights how the reward function discourages underutilization of the cache, incentivizing the agent to make more efficient use of available storage. These examples demonstrate how the reward function balances two competing objectives: maintaining high-utility (i.e., fresh and important) files in the cache and efficiently utilizing available cache memory. The agent learns to optimize both aspects through RL.

The long-term objective is to maximize the average cumulative utility of cached files while minimizing the average unused cache capacity.

5) Policy: The policy π dictates the optimal action in each state to achieve the long-term objective.

B. Proximal Policy Optimization

PPO is a widely adopted RL method, appreciated for its robustness and simplicity. The primary innovation of PPO lies in mitigating the issues of policy updates through a constrained objective, thereby ensuring more controlled updates during training.

PPO works by optimizing the expected reward while constraining the difference between the updated and previous policies. The surrogate objective used in PPO is formulated as follows:

$$L^{\text{PPO}}(\theta) = \min\left(\frac{\pi_\theta(a(t)\mid s(t))}{\pi_{\theta_{\text{old}}}(a(t)\mid s(t))}\hat{A}(t), \text{clip}\left(\frac{\pi_\theta(a(t)\mid s(t))}{\pi_{\theta_{\text{old}}}(a(t)\mid s(t))}, 1-\epsilon, 1+\epsilon\right)\hat{A}(t)\right) \tag{2}$$

where $\pi_\theta(a(t)\mid s(t))$ represents the probability of selecting action $a(t)$ in state $s(t)$ under the policy parameterized by θ, $\pi_{\theta_{\text{old}}}(a(t)\mid s(t))$ is the probability under the prior policy, $\hat{A}(t)$ is the advantage function, and ϵ is a clipping hyperparameter.

The policy is updated by maximizing this clipped surrogate objective using stochastic gradient ascent. This clipping mechanism limits the extent to which the new policy differs from the old one, maintaining a balance between policy exploration and exploitation.

The advantage function $\hat{A}(t)$ indicates the relative benefit of an action in comparison to a baseline

$$\hat{A}(t) = r(t) + \gamma^{\tau} V_{\theta'}(s') - V_{\theta'}(s) \tag{3}$$

where $V_{\theta'}(s)$ and $V_{\theta'}(s')$ represent the values of the current and next states, respectively. In standard MDPs, the state transition time is uniformly 1, leading to a fixed discount factor γ^1. However, in semi-MDPs (SMDPs), the transition time τ varies depending on the action and state dynamics. To account for this variability, the discount factor is raised to the power of τ, i.e., γ^{τ}, ensuring that future rewards are appropriately discounted over nonuniform time intervals [30], [31]. This formulation generalizes the MDP case, where $\tau = 1$ is simply a special instance.

In addition to the policy update, the value function is trained using a separate loss term, defined as the squared difference between the predicted state value $V_{\theta'}(s)$ and the target return $R(t)$

$$L^{\text{value}}(\theta') = (V_{\theta'}(s) - R(t))^2. \tag{4}$$

This loss ensures that the value network provides accurate estimates for state values during policy training.

The target return $R(t)$ is given by the one-step return

$$R(t) = r(t) + \gamma^{\tau} V_{\theta'}(s'). \tag{5}$$

This formulation ensures the value network adjusts by minimizing the error between the estimated value and the return, derived from both the immediate reward and the discounted future value.

Authorized licensed use limited to: York University. Downloaded on November 03,2026 at 23:52:02 UTC from IEEE Xplore. Restrictions apply.

C. Change Detection

Until here, we introduced our proposed caching algorithm that can learn the optimal caching policy for a given environment, considering the request rate and content popularity distribution. As discussed earlier, this learned policy may no longer remain optimal if the request rate or content popularity changes. Such changes can occur at unpredictable times, which makes it crucial to detect these shifts as quickly as possible to ensure the caching policy remains effective.

We propose our change detection mechanism for detecting changes in request rate and content popularity. One function of this mechanism focuses on monitoring fluctuations in the request rate to identify when the current pattern deviates significantly from the learned model, indicating a change. The other function targets changes in content popularity by tracking variations in the distribution of content requests. By implementing these detection functions, our approach can dynamically identify and respond to changes in the environment, allowing the caching strategy to adapt and maintain optimal performance.

1) Request Rate Change Detection: In our research work, we implement a request rate change detection function by monitoring the interarrival times of user requests. This is achieved by calculating a moving average over a specified window of length L_R, denoted as W_R. Let the interarrival times of user requests be represented by $\{\tau_1, \tau_2, \ldots, \tau_c, \ldots, \tau_C\}$, where τ_c is the time difference between the cth and $(c-1)$th request arrival. The moving average W_R over a window of size L_R is defined as

$$W_R = \frac{1}{L_R} \sum_{j=c-L_R+1}^{c} \tau_j. \tag{6}$$

This moving average W_R provides a smoothed estimate of the recent interarrival times, capturing the overall trend in request frequency. The goal is to detect any significant deviation in W_R from the average interarrival times of requests in the previous environment, denoted by $1/\lambda$, which represents the expected interarrival time under normal conditions. To achieve this, we define a threshold for acceptable variance, denoted as th_R. The condition for detecting a significant change in request rate is given by

$$|W_R - 1/\lambda| > \text{th}_R. \tag{7}$$

If the absolute difference between W_R and $1/\lambda$ exceeds th_R, it indicates a notable change in the request arrival pattern, implying that the request rate has varied significantly from the expected behavior. This condition can occur in scenarios where, for example, a sudden increase in user demand causes a spike in the request rate, or a decrease in user engagement leads to fewer incoming requests over time.

2) Content Popularity Change Detection: We propose a method for detecting changes in content popularity by keeping track of request patterns over recent time intervals. This is accomplished using a history of requests spanning the most recent L_P requests. We define a vector $W_P = \{W_1, W_2, W_3, \ldots, W_f, \ldots, W_F\}$, where W_f is the average number of times that file f is requested within the interval spanning from trials $n - L_P$ to n. Formally, W_f can be represented as

$$W_f = \frac{1}{L_P} \sum_{i=n-L_P}^{n} I_f(i) \tag{8}$$

where $I_f(i) = 1$ if file f is requested at trial i, and $I_f(i) = 0$ otherwise. Thus, W_f captures the relative popularity of file f over the most recent L_P trials. n denotes the index of the most recent request trial, and it is used to define the interval over which the average request frequency for each file is computed.

Similarly, we define W_P^{old} as the vector of average request counts for files over the previous L_P-length interval, spanning from trials $n - L_P - 1$ to $n - 1$

$$W_f^{\text{old}} = \frac{1}{L_P} \sum_{i=n-L_P-1}^{n-1} I_f(i). \tag{9}$$

Cosine Similarity for Popularity Change Detection: To detect changes in popularity, we compute the cosine similarity between W_P and W_P^{old}. Cosine similarity measures the angle between two vectors in the multidimensional space, defined as

$$\text{Cosine Similarity}\left(W_P, W_P^{\text{old}}\right) = \frac{W_P \cdot W_P^{\text{old}}}{\|W_P\| \cdot \|W_P^{\text{old}}\|} \tag{10}$$

where:

1) $W_P \cdot W_P^{\text{old}} = \sum_{f=1}^{F} W_f W_f^{\text{old}}$ is the dot product of the two vectors.
2) $\|W_P\| = \sqrt{\sum_{f=1}^{F} W_f^2}$ and $\|W_P^{\text{old}}\| = \sqrt{\sum_{f=1}^{F} (W_f^{\text{old}})^2}$ are the magnitudes (or norms) of W_P and W_P^{old}, respectively.

The cosine similarity value ranges from –1 to 1, where:

1) 1 indicates that the two vectors are identical in direction (no change in popularity).
2) 0 indicates that the vectors are orthogonal (completely different popularity patterns).
3) -1 indicates that the vectors are in opposite directions.

Rationale for Using Cosine Similarity: Cosine similarity is well-suited for popularity change detection because it captures changes in the relative proportions of requests for different files, irrespective of their absolute magnitudes. This makes it effective in distinguishing shifts in content demand patterns, even when the total request volume remains constant. For instance, if previously popular files become less requested and other files increase in popularity, the cosine similarity will capture this shift, while other measures like Euclidean distance might not be as sensitive to these relative changes.

Change Detection Mechanism: Once the cosine similarity value is computed, we maintain a moving window of these similarity values, denoted as W_C. Let $W_C = \{C_1, C_2, \ldots, C_m\}$, where C_m is the cosine similarity value calculated at the mth interval. If the average of the values in W_C falls below a predefined threshold th_C, it indicates that the similarity between W_P and W_P^{old} has decreased significantly over time, suggesting a change in the popularity distribution

$$\frac{1}{M} \sum_{m=1}^{M} C_m < \text{th}_C \tag{11}$$

Authorized licensed use limited to: York University. Downloaded on November 03,2026 at 23:52:02 UTC from IEEE Xplore. Restrictions apply.

where M is the length of the moving window. If this condition is met, we conclude that a significant change in content popularity has occurred, and the caching strategy should be updated accordingly.

This approach is efficient and responsive because it does not require prior knowledge of when or how the popularity will change, making it robust for real-world scenarios where such information is typically unavailable.

D. Transfer Learning With Demonstrations

When a change in the environment is detected, it is crucial to learn a new policy that can effectively adapt to the new dynamics. To facilitate this transition, we propose a PPO-based transfer learning mechanism that utilizes previously collected transitions from the old environment as demonstration data in the training process of the new policy.

1) Replay Buffer With Demonstration Data: The demonstration transitions are stored in the first half of the replay buffer in the new environment (Λ'), while the agent begins exploring the new environment with a high initial exploration rate ($\epsilon = 1$). During this exploration, it generates new transitions to fill the second half of the buffer. The full replay buffer provides a mix of old and new experiences. Mathematically, the buffer capacity is defined as $|\Lambda'|$, where $|\Lambda'| = 2|\Lambda|$. $|\Lambda|$ represents the size of the replay buffer before the change in the environment. As the agent continues to interact with the new environment, the oldest demonstration data is gradually overwritten by new transitions.

Overwriting the demonstration data with new transitions is beneficial for several reasons. Initially, demonstration data from the previous environment serves as a valuable starting point, stabilizing the training process by providing the agent with relevant knowledge. However, as the agent continues to explore the new environment, these old transitions become less useful because they do not accurately represent the current dynamics and may lead to suboptimal learning. By gradually replacing demonstration data with new transitions, the agent ensures that its learning focuses on the most up-to-date information, allowing it to adapt more effectively to the changes in the environment. This process strikes a balance between leveraging prior knowledge for a faster start and transitioning fully to the new environment, ultimately improving the overall policy performance and convergence speed.

To further optimize the adaptation process, we propose reinitializing the actor neural network with random values while importing the critic neural network from the previous environment and fine-tuning it. The rationale behind this approach is to promote exploration by allowing the actor to discover new actions without being biased by the outdated policy while leveraging the previously learned value function to guide learning and accelerate convergence in the new environment. This strategy balances the need for policy adaptation with value stability, ultimately speeding up the learning process.

This approach balances the exploration-exploitation trade-off by initializing the actor network for adaptability and preserving the critic's value estimations for stability. As the replay buffer updates dynamically, the algorithm effectively transitions from demonstration-driven learning to fully online learning in the new environment, thereby improving convergence speed and robustness against abrupt changes.

2) Prioritized Batch Sampling: We use the following steps to compute the priority of sampling of each transition stored in the replay buffer.

1) *Calculate Priorities:* The priority p_i of a transition is obtained as

$$p_i = \left(\text{avg}_r - r_i\right) + |\text{TDE}_i| + \varsigma \tag{12}$$

where r_i is the instant reward of transition i and it is obtained by (1). we calculate avg_r as follows:

$$\text{avg}_r = \left(\frac{\sum_{i=0}^{t} r_i}{t}\right)$$

where t represents the time step or decision epoch at which the reward $r(t)$ is computed. TDE_i is the TD error that transition i could make the last time it was sampled, and it is obtained by

$$\text{TDE}_i = V_{\theta'}(s) - R(t).$$

Moreover, ς, is a small positive constant that ensures all transitions have a chance of being sampled.

2) *Update Probabilities:* The probability of transition i being sampled is

$$P(i) = \frac{p_i^{\alpha}}{\sum_{e\in\Lambda} p_e^{\alpha}} \tag{13}$$

where the value of α determines the degree of prioritization, with $\alpha = 0$ corresponding to the uniform sampling case, i.e., transition i is sampled randomly. To adapt to environmental change, adjustments are made by applying importance sampling weights, as illustrated below

$$\omega_i = \left(\left(\frac{1}{|\Lambda|}\right) \cdot \left[\frac{1}{P(i)}\right]\right)^{\beta}$$

where β controls the degree of importance sampling. When β is 0, there is no importance sampling, whereas when β is 1, full importance sampling is employed. ω_i is multiplied by the loss function to control the impact of each transition on updating the neural network [6].

Overwriting demonstration data and using the proposed prioritized batch sampling helps with faster convergence to the optimal policy in the new environment. The rationale behind designing this prioritized batch sampling lies in its ability to emphasize more informative experiences, specifically, experiences that provide higher learning value for the current policy. By selectively sampling transitions expected to contribute most to the policy update, prioritized batch sampling guides the learning process more effectively than uniform sampling.

In dynamic environments where state-action distributions shift, certain transitions become significantly more relevant to the new optimal policy. Prioritized sampling identifies these valuable transitions, ensuring they are more frequently revisited. This targeted focus allows the agent to adapt more

Authorized licensed use limited to: York University. Downloaded on November 03,2026 at 23:52:02 UTC from IEEE Xplore. Restrictions apply.

rapidly to environmental shifts, effectively forgetting outdated information while reinforcing knowledge that aligns with the new environment's dynamics. Consequently, this mechanism accelerates the convergence toward the optimal policy, as it reduces the time and computational resources spent on learning from less relevant past experiences.

3) Actor Loss Function: We modify the loss function of the actor network in (2) as follows:

$$L^{\text{actor}}(\theta) = L^{\text{PPO}}(\theta) + \lambda_1 J_E(\pi) \tag{14}$$

where $J_E(\pi)$ is a large margin loss designed to encourage the policy π to favor actions that align with expert demonstrations, thereby enhancing the learning process in scenarios where guidance from an expert is available. $J_E(\pi)$ is defined as

$$J_E(\pi) = \max_{a \in A}[\pi_\theta(a|s) \ + l(a_E, a) \ ] - \pi_\theta(a_E|s) \tag{15}$$

where $\pi_\theta(a|s)$ denotes the probability of selecting action a given state s under the current policy parameterized by θ, and $l(a_E, a)$ is a margin penalty designed to encourage the agent to favor actions aligned with expert demonstrations. The margin penalty is defined such that it assigns a constant value for actions other than the expert action a_E, effectively boosting the adjusted probability for nonexpert actions and incentivizing the selection of a_E. The contribution of $J_E(\pi)$ is set to zero for self-generated transitions, allowing the agent to explore its environment without being overly constrained by expert behavior. This design promotes a balance between imitation of expert behavior and the flexibility to discover potentially superior strategies through exploration, enhancing the overall robustness and adaptability of the learning process.

4) Critic Loss Function: We modify the loss function of the critic network in (4) as follows:

$$L^{\text{value}}(\theta') = \mathcal{L}_{\text{1-step}} + \lambda_2 \mathcal{L}_{\text{n-step}} + \lambda_3 \mathcal{L}_{\text{reg}}. \tag{16}$$

This updated loss function incorporates different components, including 1-step and n-step TD losses, and an L2 norm regularization term. The parameters λ_2 and λ_3 control the relative contributions of each term to the overall loss function. By integrating these elements, the critic network can effectively learn from both immediate and long-term rewards, while avoiding overfitting.

The 1-step TD error, denoted as $\mathcal{L}_{\text{1-step}}$, is defined as

$$\mathcal{L}_{\text{1-step}} = (V_{\theta'}(s) - R(t))^2.$$

The n-step TD loss, represented as $\mathcal{L}_{\text{n-step}}$, is expressed as

$$\mathcal{L}_{\text{n-step}} = \left(R(t)_{(n)} + \gamma^{\tau_{(n)}} V_{\theta'}\left(s'_{(n)}\right) - V_{\theta'}(s)\right)^2 \tag{17}$$

where $V_{\theta'}(s'_{(n)})$ refers to the state encountered after n steps, and $\tau_{(n)}$ is the cumulative transition time from s to $s'_{(n)}$. The cumulative discounted reward $R(t)_{(n)}$ is calculated as

$$R(t)_{(n)} = \gamma^{\tau_{(1)}} r_{(1)} + \gamma^{\tau_{(2)}} r_{(2)} + \gamma^{\tau_{(3)}} r_{(3)} + \cdots + \gamma^{\tau_{(n)}} r_{(n)}$$
$$\tau_{(n)} = t_1 + t_2 + t_3 + \cdots + t_n$$

where t_n represents the transition time from state $s'_{(n-1)}$ to $s'_{(n)}$. $r_{(n)}$ is the immediate reward at the nth step. Incorporating n-step returns ensures that the values of subsequent states are propagated back to preceding states, enhancing the initial training process.

$\mathcal{L}_{\text{reg}}$ is an L2 regularization term designed to reduce overfitting by penalizing large weights.

Algorithms 1 and 2 are pseudocode for the proposed caching algorithm with change detection and the proposed transfer learning approaches, respectively.

Algorithm 1 Proposed Caching Algorithm With PPO and Change Detection

Input: Cache size M, File features $\mathbf{z}, \mathbf{d}, \mathbf{i}, \mathbf{l}$, Discount factor γ.
Initialize policy network π_θ and value network $V_{\theta'}$
Initialize experience buffer λ
Initialize request rate window W_R and popularity window W_P
for each time step t **do**
  Observe current state $s(t)$
  Compute memory usage: $Mem(t) = \frac{M - \sum_f \mathbf{b}(t) \cdot \mathbf{z}(t)}{M}$
  Select action $a(t) \sim \pi_\theta(\cdot|s(t))$
  if $a(t) = 1$ and cache is full **then**
    Evict file(s) with lowest utility $y_f(t)$ to make space
  end if
  Update cache status $\mathbf{b}(t)$ based on $a(t)$
  Observe next state $s(t+1)$ and compute transition time τ
  Compute reward $r(t)$ using Eq. (1)
  Store transition $(s(t), a(t), r(t), s(t+1), \tau)$ in the replay buffer
  calculate advantage $\hat{A}(t)$ and return $R(t)$
  Update policy π_θ Eq. (2)
  Update value function using Eq. (4)
  // Change Detection
  Update interarrival window W_R and content popularity window W_P
  if request rate shift detected in W_R or popularity shift detected in W_P **then**
    run Algorithm 2
  end if
end for

V. Experimental Setup and Results

This section starts by outlining the experimental setup and detailing the tools employed for implementing our proposed caching approach called transfer learning with prioritization (TLP). Following that, we present two state-of-the-art transfer learning algorithms, which will act as baseline models for performance evaluation.

We implemented our system model and trained the DRL agent using Python 3. To facilitate the creation and management of neural networks, we utilized the TensorFlow framework as mentioned in [32].

A. Configuration and Parameters

We exploit two separate neural networks for the actor and critic components. The network weights were initialized

Authorized licensed use limited to: York University. Downloaded on November 03,2026 at 23:52:02 UTC from IEEE Xplore. Restrictions apply.

Algorithm 2 Transfer Learning With Demonstrations

New replay buffer size: $|\Lambda'| = 2 \times |\Lambda|$
Initialize new replay buffer Λ' first half with Λ, second half empty
Initialize actor network weights randomly
Initialize critic network weights from old environment

while training not converged **do**
  Observe current state s
  Select action a from actor policy $\pi_\theta(a|s)$ with high exploration rate ϵ
  Execute action a, observe reward r and next state s'
  Store transition $(s(t), a(t), r(t), s(t+1), \tau)$ in second half of Λ'
  if second half of Λ' is full **then**
    Overwrite the oldest demonstration data in the first half with new transitions
  end if
  Compute priorities p_i for each transition in Λ' by Eq. (12)
  Update sampling probabilities using Eq. (13)
  Sample mini-batch from Λ' according to $P(i)$ with importance weights ω_i
  Update actor network parameters θ using loss Eq. (14)
  Update critic network parameters θ' using Eq. (16)
  Update exploration rate ϵ (gradually decrease)
end while

TABLE II
PARAMETER SETTINGS

Notation	value
F	50
γ	0.99
M	5000
β	0.6 linearly increased to 1
α	0.4
Batch size	64
$\|\Lambda\|$	2000
$\|\Lambda'\|$	4000
l for each file type	Randomly generated from [10, 30]
i for each file type	Randomly generated from [0.1, 0.9]
z for each file type	Randomly generated from [100, 1000]
λ	[0.2, 0.3]
η	[0, 1]
L_R	10
L_P	50
th_R	0.05
th_c	0.3

uniformly between -0.1 and 0.1, and the biases were set to a constant value of 0.1. The ReLU function was chosen as the activation function. Unless otherwise stated, the setting of the parameters is provided in Table II.

The file request probabilities are modeled using a Zipf distribution, governed by the parameter η within the range $0 < \eta \leq 1$. According to this distribution, the probability of requesting the fth file is expressed as $\mathfrak{p}_f = (1/\sigma f^\eta)$ [24], where σ is calculated as follows:

$$\sigma = \sum_{f=1}^{F} \frac{1}{f^\eta} \tag{18}$$

where the parameter η dictates the skewness of the Zipf distribution. As η nears 1, the probability of requesting the most frequently accessed file becomes much higher relative to other files. On the other hand, when η is closer to 0, the distribution of file popularity tends to be more uniform. The utility function is designed to grow linearly based on a file's significance and increases exponentially with its freshness.

B. Baselines

To evaluate our proposed approach, we establish baselines that encompass both change detection and transfer learning methods within RL environments.

Change Detection Baselines:

1) *Kullback–Leibler Divergence (KL Divergence):* This statistical measure the difference between two probability distributions. It computes the divergence by assessing how one distribution diverges from a second, reference distribution. Specifically, KL Divergence is defined as $D_{KL}(P||Q) = \sum_i P(i) \log([P(i)/Q(i)])$, where P and Q represent the two distributions in question. A higher KL divergence value signifies a more substantial discrepancy between the two distributions.

Transfer Learning Baselines:

1) *LFS:* Upon detecting a change in the environment, the agent resets its replay buffer and reinitializes its neural network. It then begins the process of learning the optimal policy from the ground up, without leveraging any prior knowledge from previous policies.
2) *Deep Q-Learning From Demonstrations (DQfD) [6]:* DQfD imports its policy from the previous environment and subsequently fine-tunes it within the new context. This method assigns priorities to experiences based on the TD error and retains demonstration data from earlier environments without overwriting it. DQfD is built upon the framework of double deep Q-learning.
3) *Q-Value-Based Deep Transfer RL (QDTRL) [7]:* Like DQfD, QDTRL is grounded in the principles of DDQL, but it adopts a different approach to transfer learning. This method does not import demonstration data from prior environments and instead samples experience batches randomly without any prioritization. QDTRL incorporates the Q-values derived from the previous environment's policy as an additional term in its loss function, effectively using these Q-values as prior knowledge to guide the learning process.

C. Evaluation Criteria

The performance of our proposed approach is assessed using the following evaluation criteria.

1) *Detected Change Point:* One of the key goals is for the agent to detect the change point in the environment as promptly as possible. Early detection allows the agent to transition swiftly into the transfer learning phase, thereby enabling it to adapt its policy to the new environment more efficiently. The effectiveness of different

Authorized licensed use limited to: York University. Downloaded on November 03,2026 at 23:52:02 UTC from IEEE Xplore. Restrictions apply.

change detection methods is measured by how quickly they identify changes in the environment. The faster the change point is detected, the less time is wasted on the outdated policy, resulting in improved adaptability and overall system performance. The algorithm that minimizes this detection delay is considered the most effective for change detection.

2) *Rate of Convergence:* Once the agent detects a change in the environment and initiates the transfer learning process, the rate at which it converges to the optimal policy becomes crucial. The convergence rate refers to how rapidly the agent stabilizes at its peak average reward in the new environment. A high rate of convergence is indicative of the algorithm's efficiency in learning and adapting to new conditions.
3) *Average Reward:* For each training trial, the average of all instantaneous rewards accumulated up to that point is computed. The primary objective of an RL agent operating in an infinite horizon is to maximize this average reward over time. Since a higher reward corresponds to a higher cache hit rate, an algorithm that achieves a higher long-term average reward demonstrates superior performance.

D. Experimental Findings

In this section, we present our experimental results and provide a comprehensive comparison with the baseline methods. We first analyze the performance of the change detection mechanism, focusing on its ability to promptly identify shifts in the environment. This evaluation emphasizes detection speed and accuracy, benchmarking our approach against traditional methods to highlight its effectiveness. Next, we shift our attention to the transfer learning algorithms, examining their ability to quickly adapt and stabilize in the new environment once a change has been detected. We compare their convergence rates and learning efficiency against other benchmark algorithms, demonstrating how our proposed method outperforms existing solutions in terms of adaptability and overall performance.

1) Performance evaluation of change detection mechanism: In this section, we present the results of our proposed change detection mechanism designed to detect changes in content popularity. The change in file popularity occurs at trial 1000. Table III shows the specific trials at which our change detection mechanism detected these changes under different experimental scenarios. We varied the number of file types (50, 20, and 10) and considered three distinct cases: 1) exchanging the popularity of the most and the least popular files; 2) exchanging the popularity of two popular files with two less popular files; and 3) exchanging the popularity of five popular files with five less popular files.

Table III showcases the performance of our change detection mechanism using cosine similarity under various scenarios. The results indicate that setting the threshold (th_c) to 0.5 is problematic, as it leads to premature detections well before the actual change point at trial 1000. For example, in multiple cases, the algorithm incorrectly identifies a change as early as trial 51, highlighting the need for a lower threshold value.

TABLE III
CHANGE DETECTION RESULTS FOR COSINE SIMILARITY WITH DIFFERENT THRESHOLDS

Change Scenario	$\mathbf{th_c}$	$\mathbf{F = 50}$	$\mathbf{F = 20}$	$\mathbf{F = 10}$
2 Popular and 2 Less Popular Files Exchanged	0.2	1056	1060	1063
	0.3	1044	1041	1041
	0.4	701	1024	1033
	0.5	51	51	51
Most and Least Popular Files Exchanged	0.2	1049	1046	1052
	0.3	1042	1035	1040
	0.4	701	1023	1032
	0.5	51	51	51
5 Popular and 5 Less Popular Files Exchanged	0.2	1045	1041	1047
	0.3	1040	1031	1037
	0.4	701	1021	1029
	0.5	51	51	51

TABLE IV
CHANGE DETECTION RESULTS FOR KL DIVERGENCE WITH DIFFERENT THRESHOLDS

Change Scenario	Threshold	$\mathbf{F = 50}$	$\mathbf{F = 20}$	$\mathbf{F = 10}$
2 Popular and 2 Less Popular Files Exchanged	2	51	51	736
	4	73	1047	1055
	8	1061	1063	1055
	10	1473	–	–
Most and Least Popular Files Exchanged	2	51	51	736
	4	73	1047	1055
	8	1155	1065	1055
	10	1048	1126	1419
5 Popular and 5 Less Popular Files Exchanged	2	51	51	736
	4	73	1027	1055
	8	1049	1072	1287
	10	1048	1050	1055

Reducing the threshold to 0.4 enhances detection accuracy for environments with 10 and 20 file types, allowing the algorithm to accurately detect changes within 23–33 trials after the true change point. However, for environments with 50 file types, this threshold still triggers false alarms. In contrast, a threshold of 0.3 delivers the best performance across all file types and scenarios, achieving reliable detection without introducing false positives, making it the most suitable option for identifying shifts in popularity trends.

Table IV illustrates the results of our change detection algorithm utilizing KL divergence. Selecting an appropriate threshold for KL divergence is inherently complex, as it is largely contingent upon the specific differences between the two environments. For example, when the change involves the popularity of five popular files and five less popular files, a threshold of ten performs satisfactorily across all file types. Conversely, in scenarios where the change consists of the popularity of the most and least popular files, a threshold of 10 is ineffective. In such cases, the algorithm may fail to detect the change entirely or only identify it after trial 473, which is too late. In practical, real-world situations, there is often a lack of prior knowledge regarding the exact nature of the differences between environments. This uncertainty makes KL divergence susceptible to inaccuracies, thereby rendering it a less reliable choice for change detection in this context.

Authorized licensed use limited to: York University. Downloaded on November 03,2026 at 23:52:02 UTC from IEEE Xplore. Restrictions apply.

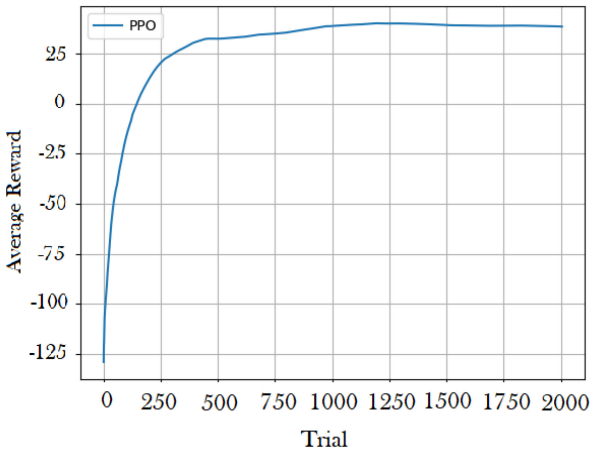


Fig. 2. Convergence of the proposed caching approach in the initial environment.

Based on the results in Tables III and IV, while both methods can effectively detect changes in file popularity, cosine similarity enables the use of a universal threshold across different numbers of file types, whereas KL Divergence requires adjusting the threshold for each case.

E. Transfer learning adaptation and comparison

In this section, we consider two distinct types of changes in the environment to thoroughly evaluate the robustness and adaptability of the proposed algorithms. The first type focuses on a scenario where only the content popularity changes, keeping the overall request rate constant. The second type explores a scenario where content popularity remains unchanged but the request rate changes. Before diving into these dynamic scenarios, we first demonstrate the convergence of the caching approach in the initial environment, as illustrated in Fig. 2.

1) *Performance Comparison Under Popularity Change:* In the first experiment, the popularity of the most and second-most popular files is swapped with the least and second-least popular files, while maintaining the total request rate unchanged. This setting aims to evaluate how the algorithms respond to a shift in content popularity without altering the overall user demand pattern. Fig. 3(a) and (b) present the outputs of our mechanism for popularity change detection and request rate change detection functions, respectively. As expected, the popularity change detection mechanism promptly identifies the swap, as the drastic reordering of popularities alters the request frequencies for these specific files. However, the request rate change detection mechanism correctly does not signal any changes, since the request rate remains unaltered. This confirms that the request rate detection mechanism is well-calibrated and does not produce false positives when only popularity changes are observed. Fig. 4 illustrates the average reward over time for four algorithms: 1) LFS; 2) DQfD; 3) QDTRL; and 3) TLP. LFS begins LFS and lacks any prior knowledge to assist in the new environment. Without any information transfer, LFS has to explore the environment extensively before it can identify an optimal

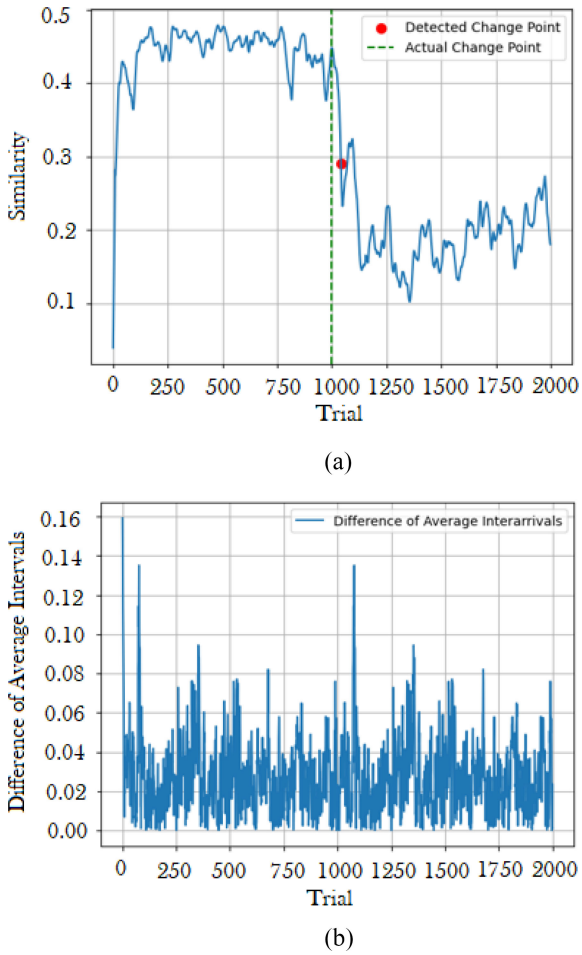


Fig. 3. Environment change detection results under popularity shift: (a) Popularity shift detection, and (b) Request rate variation detection.

caching strategy, leading to a prolonged convergence period. The performance of QDTRL is comparable to that of LFS, showing only marginal improvements. This is likely because QDTRL still struggles when the changes are substantial, rendering much of the previously learned information less useful. This suggests that QDTRL might require more sophisticated adaptation mechanisms to effectively handle severe environmental changes and benefit from the prior knowledge. DQfD outperforms both LFS and QTDRL in terms of achieving faster convergence. The primary reason is its ability to effectively leverage expert demonstrations to guide early-stage learning, reducing the exploration time needed to identify optimal policies. On the other hand, TLP significantly outperforms all other baselines and converges much faster. TLP outperforms the other baselines primarily because of the way it prioritizes transitions, which enables it to demonstrate more stabilized behavior in response to popularity changes. By effectively recognizing and assigning higher importance to critical transitions, TLP adapts more efficiently to the new environment. Furthermore, TLP's ability to relearn its action selection policy while leveraging knowledge from the source environment allows it to strike a balance

Authorized licensed use limited to: York University. Downloaded on November 03,2026 at 23:52:02 UTC from IEEE Xplore. Restrictions apply.

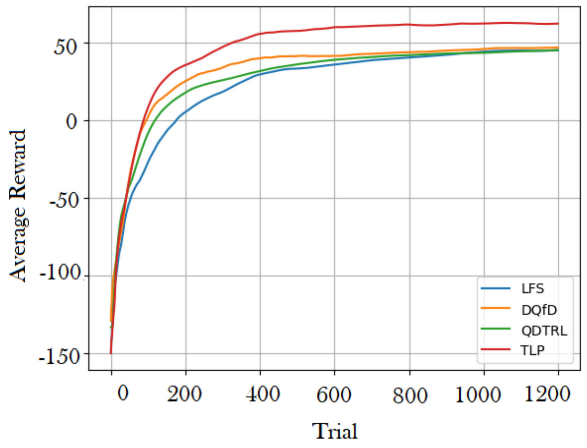
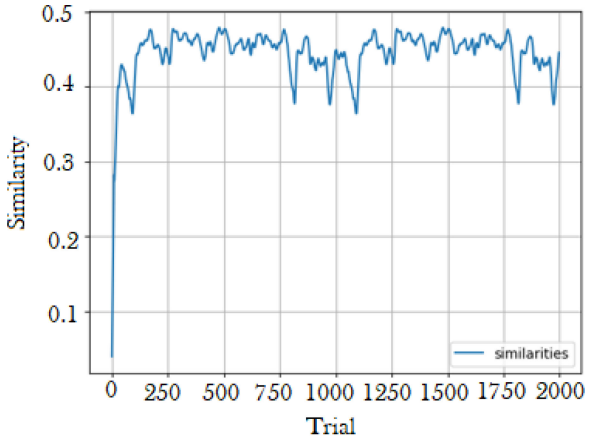


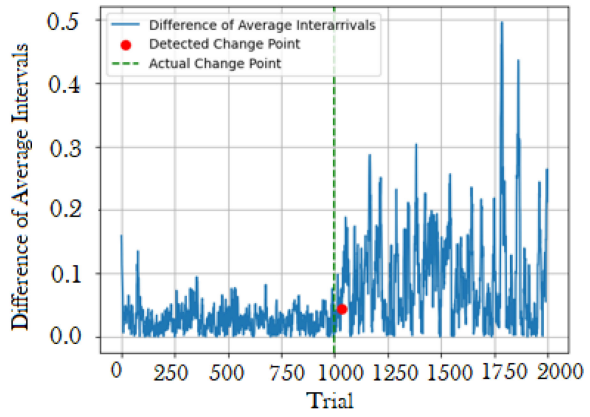
Fig. 4. Average reward over time in the new environment under popularity change.

between exploration and exploitation. This is particularly advantageous in scenarios where the popularity of files changes drastically, as it can exploit the most relevant transitions while exploring new patterns. Another key advantage of TLP is its gradual overwriting of demonstration data. Unlike DQfD and QDTRL, which suffer when the differences in popularity distributions between environments are too high, TLP can progressively phase out outdated information, making it robust to significant shifts in file popularity and ensuring improved performance even under abrupt changes. This makes TLP not only adaptable to the new environment but also highly effective in retaining critical knowledge while adjusting its policy accordingly.

2) *Performance Comparison Under Request Rate Change:* In this section, we consider a scenario where the request rate changes while the popularity distribution of the content remains constant. Before change $\lambda = 5$, and after change $\lambda = 3.3$. Fig. 5(a) and (b) show the outputs of the popularity and request rate change detection functions, respectively. As expected, the popularity change detection algorithm does not detect any change, confirming that the content popularity remains stable. On the other hand, the request rate change detection algorithm promptly identifies the change in request rate, highlighting its sensitivity to variations in request patterns. Fig. 6 illustrates the performance of TLP and baseline algorithms in the new environment after the change. LFS consistently performs the worst, as it lacks the ability to transfer knowledge from the previous environment and must start LFS. DQfD is closely aligned with LFS, likely due to its reliance on outdated demonstration data that is not adapted to the new request rate pattern. This similarity arises because DQfD struggles to adapt its policy when the distribution of requests shifts, causing it to converge slowly in response to the change. QDTRL, in contrast, performs better than both LFS and DQfD, as it partially transfers knowledge from the previous environment. However, its improvement is limited because it does



(a)



(b)

Fig. 5. Environment change detection results under request rate change: (a) Popularity shift detection, and (b) Request rate variation detection.

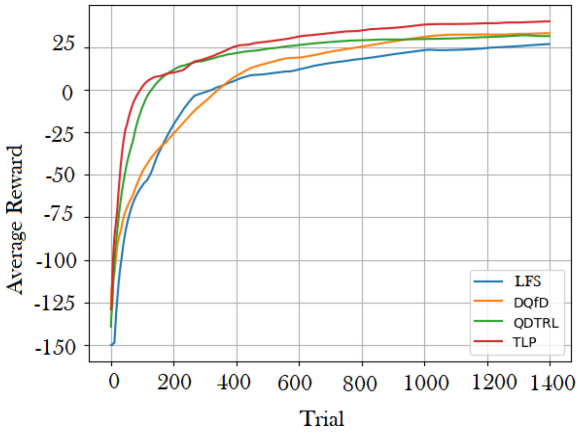


Fig. 6. Average reward over time in the new environment under request rate change.

not fully account for request rate variations, making its adaptation slower. TLP significantly outperforms all other baselines and converges quickly in the new environment, for the same reason stated earlier. More importantly, TLP explicitly models the request rate using an SMDP formulation. This structured approach

Authorized licensed use limited to: York University. Downloaded on November 03,2026 at 23:52:02 UTC from IEEE Xplore. Restrictions apply.

allows the algorithm to better recognize changes in the request rate, leading to more informed decision-making. By integrating SMDP, TLP achieves a balance between utilizing existing knowledge and exploring the new environment, allowing it to adjust efficiently and maintain a high average reward even when the request dynamics shift. The results demonstrate that our proposed method can promptly detect changes in content popularity and request arrival rates, enabling the system to rapidly adapt its caching policy. This capability is especially important in practical deployments where user behavior and content demand are highly dynamic. For example, during events, viral trends, or sudden shifts in network usage patterns. From a system design perspective, the results highlight the importance of integrating lightweight change detection modules that continuously monitor request statistics without introducing significant computational overhead. Moreover, our use of transfer learning shows that previously learned policies can be reused to accelerate learning in new environments, reducing the time and resources needed for retraining. This suggests that practical caching systems can maintain high performance by combining detection-driven policy updates with experience reuse strategies, making them more scalable and responsive to change.

VI. Conclusion

In conclusion, this article presents an effective edge caching strategy that addresses the challenges of dynamic environments by integrating key file attributes and accounting for random file request arrivals. By incorporating PPO and developing an effective mechanism for detecting changes in content popularity and request rates, the proposed solution adapts efficiently to variations in the environment, avoiding the computational costs of learning new policies from scratch. The transfer learning-based PPO algorithm further accelerates convergence by leveraging prior knowledge, leading to faster adaptation and improved performance. Simulation results confirm that our approach can promptly and effectively detect environment changes, and the proposed transfer learning approach significantly outperforms recent transfer learning-based methods in terms of convergence, demonstrating its potential to enhance edge caching in dynamic real-world scenarios.

Future work can explore several directions to further improve the system. One promising avenue is extending the algorithm to multiagent edge caching environments, where multiple edge nodes cooperate or learn in parallel. Another direction involves incorporating user mobility patterns and contextual information to enable predictive and proactive caching strategies. Additionally, the integration of energy-efficiency objectives would make the approach more suitable for resource-constrained edge devices. Finally, applying meta-RL techniques could enable the system to generalize across multiple dynamic environments, further reducing the need for retraining and enhancing adaptability.

References

[1] J. Gao, S. Zhang, L. Zhao, and X. Shen, "The design of dynamic probabilistic caching with time-varying content popularity," *IEEE Trans. Mobile Comput.*, vol. 20, no. 4, pp. 1672–1684, Apr. 2021.

[2] T. Zong, C. Li, Y. Lei, G. Li, H. Cao, and Y. Liu, "Cocktail edge caching: Ride dynamic trends of content popularity with ensemble learning," *IEEE/ACM Trans. Netw.*, vol. 31, no. 1, pp. 208–219, Feb. 2023.

[3] A. Krishna, R. Burra, and C. Singh, "Caching dynamic contents with varying popularity," in *Proc. 19th Int. Symp. Model. Optim. Mobile, Ad Hoc, Wireless Netw. (WiOpt)*, 2021, pp. 1–8.

[4] Z. Zhang and M. Tao, "Deep learning for wireless coded caching with unknown and time-variant content popularity," *IEEE Trans. Wireless Commun.*, vol. 20, no. 2, pp. 1152–1163, Feb. 2021.

[5] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," 2017, *arXiv:1707.06347*.

[6] T. Hester et al., "Deep Q-learning from demonstrations," in *Proc. AAAI Conf. Artif. Intell.*, 2018, pp. 3223–3230.

[7] H. Zhou, M. Erol-Kantarci, and H. V. Poor, "Learning from peers: Deep transfer reinforcement learning for joint radio and cache resource allocation in 5G RAN slicing," *IEEE Trans. Cogn. Commun. Netw.*, vol. 8, no. 4, pp. 1925–1941, Dec. 2022.

[8] C. Sun, X. Li, J. Wen, X. Wang, Z. Han, and V. C. M. Leung, "Federated deep reinforcement learning for recommendation-enabled edge caching in mobile edge-cloud computing networks," *IEEE J. Sel. Areas Commun.*, vol. 41, no. 3, pp. 690–705, Mar. 2023.

[9] H. Wu, A. Nasehzadeh, and P. Wang, "A deep reinforcement learning-based caching strategy for IoT networks with transient data," *IEEE Trans. Veh. Technol.*, vol. 71, no. 12, pp. 13310–13319, Dec. 2022.

[10] X. Zhou et al., "Edge computation offloading with content caching in 6G-enabled IoV," *IEEE Trans. Intell. Transp. Syst.*, vol. 25, no. 3, pp. 2733–2747, Mar. 2024.

[11] H. Zhu, Y. Cao, X. Wei, W. Wang, T. Jiang, and S. Jin, "Caching transient data for Internet of Things: A deep reinforcement learning approach," *IEEE Internet Things J.*, vol. 6, no. 2, pp. 2074–2083, Apr. 2019.

[12] H. Zhou, H. Wang, Z. Yu, G. Bin, M. Xiao, and J. Wu, "Federated distributed deep reinforcement learning for recommendation-enabled edge caching," *IEEE Trans. Services Comput.*, vol. 17, no. 6, pp. 3640–3656, Nov./Dec. 2024.

[13] T. Wang et al., "Towards intelligent adaptive edge caching using deep reinforcement learning," *IEEE Trans. Mobile Comput.*, vol. 23, no. 10, pp. 9289–9303, Oct. 2024.

[14] C. Zhong, M. C. Gursoy, and S. Velipasalar, "Deep reinforcement learning-based edge caching in wireless networks," *IEEE Trans. Cogn. Commun. Netw.*, vol. 6, no. 1, pp. 48–61, Mar. 2020.

[15] Y. Liu, J. Jia, J. Cai, and T. Huang, "Deep reinforcement learning for reactive content caching with predicted content popularity in three-tier wireless networks," *IEEE Trans. Netw. Service Manag.*, vol. 20, no. 1, pp. 486–501, Mar. 2023.

[16] J. Yao and N. Ansari, "Caching in dynamic IoT networks by deep reinforcement learning," *IEEE Internet Things J.*, vol. 8, no. 5, pp. 3268–3275, Mar. 2021.

[17] S. Gao, P. Dong, Z. Pan, and G. Y. Li, "Reinforcement learning based cooperative coded caching under dynamic popularities in ultra-dense networks," *IEEE Trans. Veh. Technol.*, vol. 69, no. 5, pp. 5442–5456, May 2020.

[18] Q. Fan, X. Li, J. Li, Q. He, K. Wang, and J. Wen, "PA-cache: Evolving learning-based popularity-aware content caching in edge networks," *IEEE Trans. Netw. Service Manag.*, vol. 18, no. 2, pp. 1746–1757, Jun. 2021.

[19] S. Müller, O. Atan, M. Van Der Schaar, and A. Klein, "Smart caching in wireless small cell networks via contextual multi-armed bandits," in *Proc. IEEE Int. Conf. Commun. (ICC)*, 2016, pp. 1–7.

[20] X. Wei, J. Liu, Y. Wang, C. Tang, and Y. Hu, "Wireless edge caching based on content similarity in dynamic environments," *J. Syst. Archit.*, vol. 115, May 2021, Art. no. 102000.

[21] S. Li, J. Xu, M. Van Der Schaar, and W. Li, "Popularity-driven content caching," in *Proc. 35th Annu. IEEE Int. Conf. Comput. Commun. (INFOCOM)*, 2016, pp. 1–9.

[22] A. Nasehzadeh and P. Wang, "A deep reinforcement learning-based caching strategy for Internet of Things," in *Proc. IEEE/CIC Int. Conf. Commun. China (ICCC)*, 2020, pp. 969–974.

[23] S. Vural, N. Wang, P. Navaratnam, and R. Tafazolli, "Caching transient data in Internet content routers," *IEEE/ACM Trans. Netw.*, vol. 25, no. 2, pp. 1048–1061, Apr. 2017.

Authorized licensed use limited to: York University. Downloaded on November 03,2026 at 23:52:02 UTC from IEEE Xplore. Restrictions apply.

[24] H. Gomaa, G. G. Messier, C. Williamson, and R. Davies, "Estimating instantaneous cache hit ratio using Markov chain analysis," *IEEE/ACM Trans. Netw.*, vol. 21, no. 5, pp. 1472–1483, Oct. 2013.
[25] M. L. Puterman, *Markov Decision Processes: Discrete Stochastic Dynamic Programming*. Hoboken, NJ, USA: Wiley, 2014.
[26] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*. Cambridge, MA, USA: MIT Press, 2018.
[27] F. Niknia, P. Wang, A. Agarwal, and Z. Wang, "Edge caching based on deep reinforcement learning," in *Proc. IEEE/CIC Int. Conf. Commun. China (ICCC)*, 2023, pp. 1–6.
[28] Q. N. Nguyen et al., "PPCS: A progressive popularity-aware caching scheme for edge-based cache redundancy avoidance in information-centric networks," *Sensors*, vol. 19, no. 3, p. 694, Feb. 2019.
[29] F. Niknia, P. Wang, Z. Wang, A. Agarwal, and A. S. Rezaei, "Attention-enhanced prioritized proximal policy optimization for adaptive edge caching," *IEEE Trans. Veh. Technol.*, early access, Apr. 2, 2025, doi: 10.1109/TVT.2025.3552084.
[30] P. Poupart (Univ. Waterloo, Waterloo, ON, Canada). *Semi-Markov Decision Processes (SMDPs)*. Accessed: Jun. 21, 2025. [Online]. Available: https://cs.uwaterloo.ca/∼ppoupart/teaching/cs885-spring18/slides/cs885-lecture15c.pdf
[31] F. Niknia, V. Hakami, and K. Rezaee, "An SMDP-based approach to thermal-aware task scheduling in NoC-based MPSoC platforms," *J. Parallel Distrib. Comput.*, vol. 165, pp. 79–106, Jul. 2022.
[32] T. Developers, "Tensorflow." Zenodo, 2022. Accessed: Jun. 21, 2025. [Online]. Available: https://ui.adsabs.harvard.edu/abs/2021zndo...4758419D/abstract



Farnaz Niknia received the B.Sc. (Exceptional Talent) degree from the University of Tabriz, Tabriz, Iran, the M.Sc. (Exceptional Talent) degree from Iran University of Science and Technology, Tehran, Iran. She is currently pursuing the Ph.D. degree in electrical engineering and computer science with York University, Toronto, Canada.

Her research interests include reinforcement learning, edge computing, and learning in dynamic environments.



Ping Wang (Fellow, IEEE) is a Professor with the Department of Electrical Engineering and Computer Science, York University, Toronto, Canada, where she is a Tier 2 York Research Chair. Prior to that, she was with Nanyang Technological University, Singapore, from 2008 to 2018. Her scholarly works have been widely disseminated through top-ranked IEEE journals/conferences. Her recent research interests focus on integrating artificial intelligence techniques into communications networks.

She has been serving as the Associate Editor-in-Chief for IEEE COMMUNICATIONS SURVEYS & TUTORIALS and an Editor for several reputed journals, including IEEE TRANSACTIONS ON WIRELESS COMMUNICATIONS and IEEE TRANSCATIONS ON NETWORK SCIENCE AND ENGINEERING. She is a Distinguished Lecturer of the IEEE Vehicular Technology Society. She is also the Chair of the Education Committee of IEEE VTS.

Authorized licensed use limited to: York University. Downloaded on November 03,2026 at 23:52:02 UTC from IEEE Xplore. Restrictions apply.